



Published in YML Innovation Lab



Prasad Pai

Follow

Aug 7, 2018 · 7 min read · Listen



Save



Sign in to Medium with Google



Sandipan Dey

sandipan.dey@gmail.com

Continue as Sandipan

To create your account, Google will share your name, email address, and profile picture with Medium. See Medium's [privacy policy](#) and [terms of service](#).

How to use Dataset and Iterators in Tensorflow with code samples

From the time I have started using Tensorflow, I have always been feeding the data to my graph during training, testing or inferencing using the feed_dict mechanism of Session. This particular practice has been advised by Tensorflow developers to be strongly discontinued either during the training or repeatedly testing same series of dataset. The only particular scenario in which feed_dict mechanism is to be used is during inferencing of data during deployment. The replacement of feed_dict has taken place with Dataset and Iterator. The dataset can be created either with Numpy array or TFRecords or with text.

In this post, we will be exploring on Datasets and Iterators. We will start with how to create datasets using some source data and then apply various type of transformations to it. We will demonstrate on how to do training using various types of iterators with MNIST handwritten digits data on LeNet-5 model.

Note: The Tensorflow Dataset class can get very confusing with word meant for datasets like X_train, y_train etc. Hence, going forward in this article, I am referring 'Dataset' (capital D) as Tensorflow Dataset class and 'dataset' as dataset of X_train, y_train etc.

Datasets Creation

Datasets can be generated using multiple type of data sources like Numpy, TFRecords, text files, CSV files etc. The most commonly used practice for

generating Datasets is from Numpy (or Tensors). Lets go through each of the functions provided by Tensorflow to generate them.

a) **from_tensor_slices**: This method accepts individual (or multiple) Numpy (or Tensors) objects. In case you are feeding multiple objects, pass them as tuple and make sure that all the objects have same size in zeroth dimension.

```
1  # Assume batch size is 1
2  dataset1 = tf.data.Dataset.from_tensor_slices(tf.range(10, 15))
3  # Emits data of 10, 11, 12, 13, 14, (One element at a time)
4
5  dataset2 = tf.data.Dataset.from_tensor_slices((tf.range(30, 45, 3), np.arange(60, 70, 2)))
6  # Emits data of (30, 60), (33, 62), (36, 64), (39, 66), (42, 68)
7  # Emits one tuple at a time
8
9  dataset3 = tf.data.Dataset.from_tensor_slices((tf.range(10), np.arange(5)))
10 # Dataset not possible as zeroth dimension is different at 10 and 5
```

CreateDataset.py hosted with ❤ by GitHub

[view raw](#)

Code sample showing Dataset creation with from_tensor_slices

b) **from_tensors**: Just like from_tensor_slices, this method also accepts individual (or multiple) Numpy (or Tensors) objects. But this method doesn't support batching of data, i.e all the data will be given out instantly. As a result, you can pass differently sized inputs at zeroth dimension if you are passing multiple objects. This method is useful in cases where dataset is very small or your learning model needs all the data at once.

```
1  dataset1 = tf.data.Dataset.from_tensors(tf.range(10, 15))
2  # Emits data of [10, 11, 12, 13, 14]
3  # Holds entire list as one element
4
5  dataset2 = tf.data.Dataset.from_tensors((tf.range(30, 45, 3), np.arange(60, 70, 2)))
6  # Emits data of ([30, 33, 36, 39, 42], [60, 62, 64, 66, 68])
7  # Holds entire tuple as one element
8
9  dataset3 = tf.data.Dataset.from_tensors((tf.range(10), np.arange(5)))
10 # Possible with from_tensors, regardless of zeroth dimension mismatch of constituent elements.
11 # Emits data of ([1, 2, 3, 4, 5, 6, 7, 8, 9], [0, 1, 2, 3, 4])
12 # Holds entire tuple as one element
```

Code sample showing Dataset creation with from_tensors

c) **from_generator**: In this method, a generator function is passed as input. This method is useful in cases where you wish to generate the data at runtime and as such no raw data exists with you or in scenarios where your training data is extremely huge and it is not possible to store them in your disk. I would strongly encourage people to **not use** this method for the purpose of generating data augmentations.

```
1  # Assume batch size is 1
2  def generator(sequence_type):
3      if sequence_type == 1:
4          for i in range(5):
5              yield 10 + i
6      elif sequence_type == 2:
7          for i in range(5):
8              yield (30 + 3 * i, 60 + 2 * i)
9      elif sequence_type == 3:
10         for i in range(1, 4):
11             yield (i, ['Hi'] * i)
12
13  dataset1 = tf.data.Dataset.from_generator(generator, (tf.int32), args = ([1]))
14  # Emits data of 10, 11, 12, 13, 14, (One element at a time)
15
16  dataset2 = tf.data.Dataset.from_generator(generator, (tf.int32, tf.int32), args = ([2]))
17  # Emits data of (30, 60), (33, 62), (36, 64), (39, 66), (42, 68)
18  # Emits one tuple at a time
19
20  dataset3 = tf.data.Dataset.from_generator(generator, (tf.int32, tf.string), args = ([3]))
21  # Emits data of (1, ['Hi']), (2, ['Hi', 'Hi']), (3, ['Hi', 'Hi', 'Hi'])
22  # Emits one tuple at a time
```

CreateDataset.py hosted with ❤ by GitHub

[view raw](#)

Code sample showing Dataset creation with from_generator

Datasets Transformations

Once you have created the Dataset covering all the data (or scenarios, in some cases like, runtime data generation), it is time to apply various types of transformation. Let us go through some of commonly used transformations.

a) **Batch**: Batch corresponds to sequentially dividing your dataset by the specified batch size.

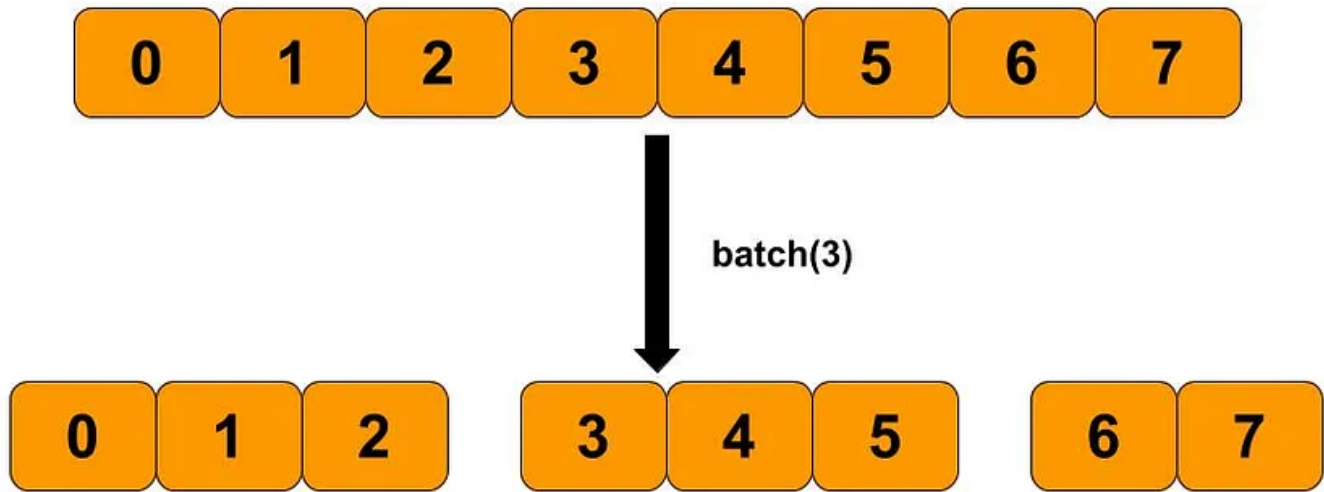


Illustration of batch transformation

b) **Repeat**: Whatever Dataset you have generated, use this transformation to create duplicates of the existing data in your Dataset.

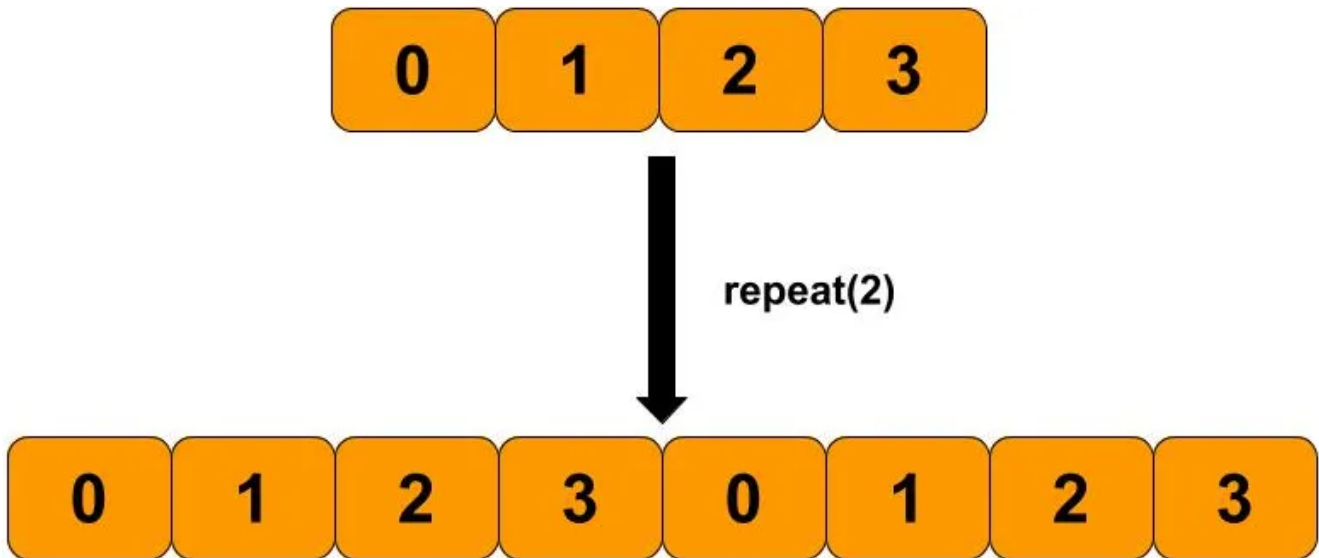


Illustration of repeat transformation

c) **Shuffle**: Shuffle transformation randomly shuffles the data in your Dataset.

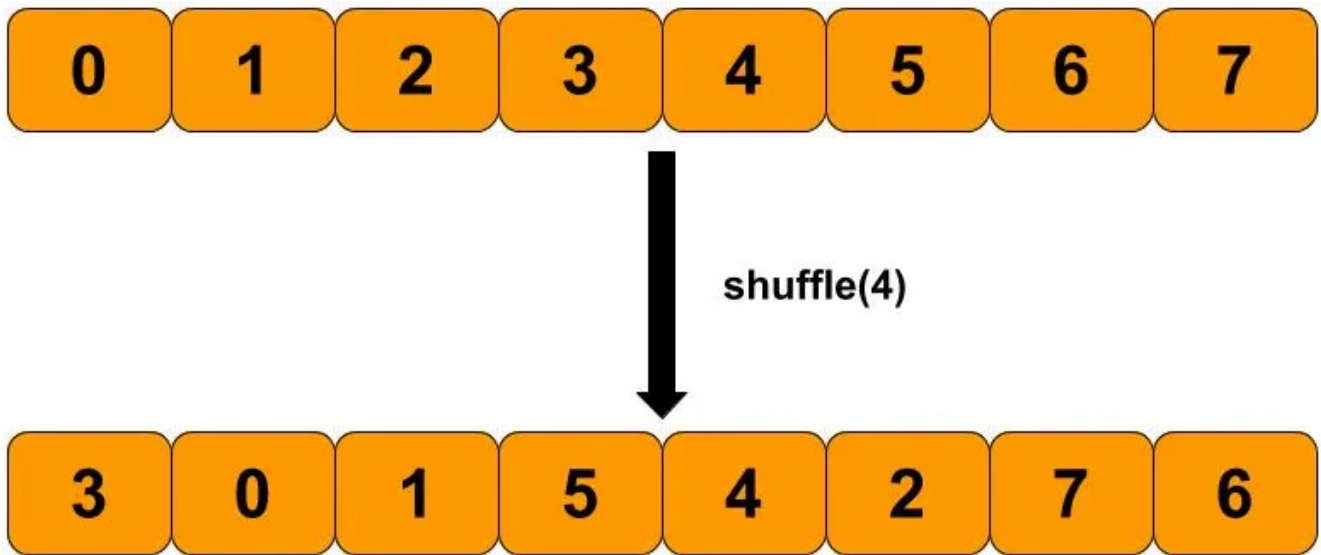


Illustration of shuffle transformation

d) **Map**: In Map transformation, you can apply some operations to all the individual data elements in your dataset. Use this particular transformation to apply various types of data augmentation. (You can check my other [Medium post on image augmentation](#)).

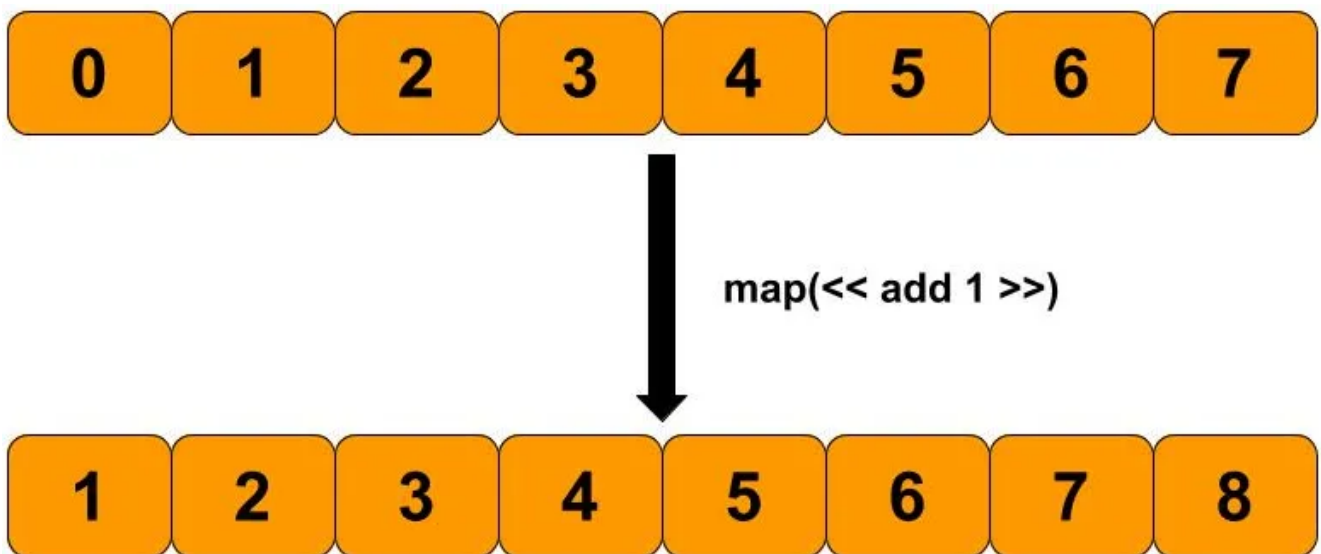


Illustration showing map transformation

e) **Filter**: During the course of training, if you wish to filter out some elements from Dataset, use filter function.

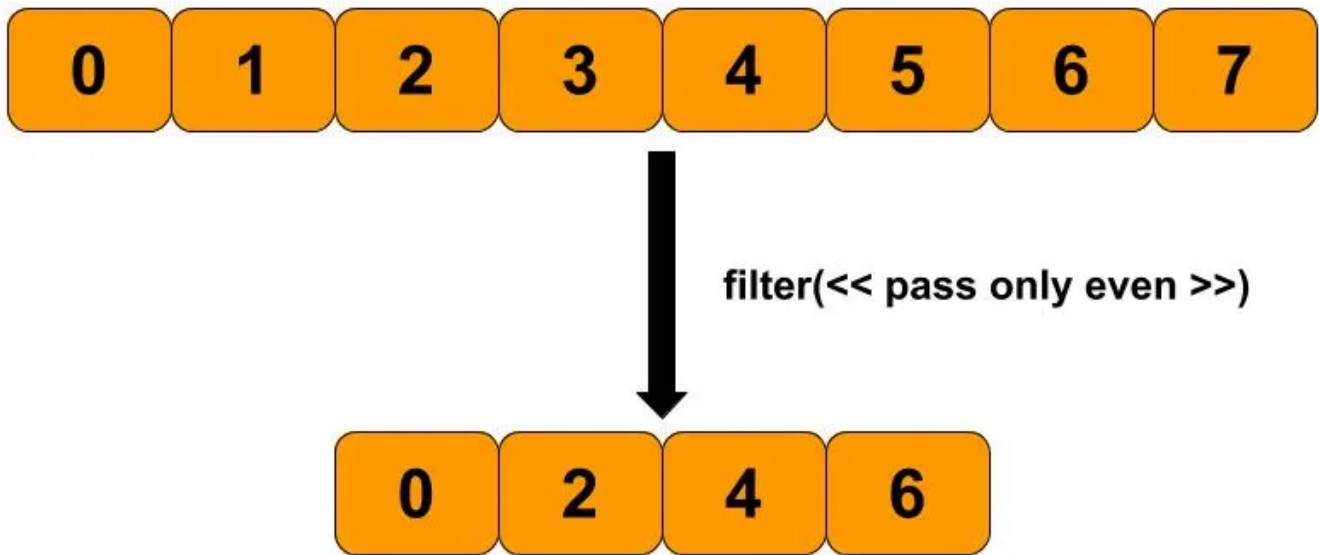


Illustration showing filter transformation

The code example of various transformations being applied on a Dataset is shown next.

```
1 dataset = tf.data.Dataset.from_tensor_slices(tf.range(10))
2 # Create a dataset with data of [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
3
4 dataset = dataset.repeat(2)
```

Open in app [↗](#)

Sign up

Sign In

```
9 # Shuffle the dataset
10 # Assumed shuffling: [3, 0, 7, 9, 4, 2, 5, 0, 1, 7, 5, 9, 4, 6, 2, 8, 6, 8, 1, 3]
11
12 def map_fn(x):
13     return x * 3
14
15 dataset = dataset.map(map_fn)
16 # Same as dataset = dataset.map(lambda x: x * 3)
17 # Multiply each element with 3 using map transformation
18 # Dataset: [9, 0, 21, 27, 12, 6, 15, 0, 3, 21, 15, 27, 12, 18, 6, 24, 18, 24, 3, 9]
19
20 def filter_fn(x):
21     return tf.reshape(tf.not_equal(x % 5, 1), [])
22
23 dataset = dataset.filter(filter_fn)
24 # Same as dataset = dataset.filter(lambda x: tf.reshape(tf.not_equal(x % 5, 1), []))
25 # Filter out all those elements whose modulus 5 returns 1
26 # Dataset: [9, 0, 27, 12, 15, 0, 3, 15, 27, 12, 18, 24, 18, 24, 3, 9]
27
28 dataset = dataset.batch(4)
```

```
29 # Batch at every 4 elements
30 # Dataset: [9, 0, 27, 12], [15, 0, 3, 15], [27, 12, 18, 24], [18, 24, 3, 9]
```

DatasetTransformation.py hosted with ❤️ by GitHub

[view raw](#)

The ordering of the application of the transformation is very important. Your model may learn differently for the same Dataset but differently ordered transformations. Take a look at the code sample in which it has been shown that different set of data is produced.



1.5K



9

Code showing different datasets produced with different order of transformations

Building LeNet-5 Model

Before we start the iterators part, let us quickly build our LeNet-5 Model and extract the MNIST data. I have used [Tensorflow's Slim library](#) to build the model in few

lines. This is going to be the common code for all types of iterators we are going to work on next.

Common code of LeNet-5 model and MNIST data extraction

Iterators

Now, let's start building up the iterators. Tensorflow has provided four types of iterators and each of them has a specific purpose and use-case behind it.

Regardless of the type of iterator, get_next function of iterator is used to create an operation in your Tensorflow graph which when run over a session, returns the values from the fed Dataset of iterator. Also, iterator doesn't keep track of how many elements are present in the Dataset. Hence, it is normal to keep running the

iterator's `get_next` operation till Tensorflow's `tf.errors.OutOfRangeError` exception is occurred. This is usually the skeleton code of how a Dataset and iterator looks like.

Skeleton code of Dataset and Iterators

Next, let's look into each type of iterator.

a) One-shot iterator

This is the most basic type of iterator. All the data with all types of transformations that is needed in the dataset has to be decided before the Dataset is fed into this iterator. One-shot iterator will iterate through all the elements present in Dataset and once exhausted, cannot be used anymore. As a result, the Dataset generated for this iterator can tend to occupy a lot of memory.

Code sample showing One-shot iterator pipeline

In the example above, we have generated the Dataset for a total of 10 epochs. Use this particular iterator only if your dataset is small in size or in cases where you would like to perform testing on your model only once.

b) Initializable

In One-shot iterator, we had the shortfall of repetition of same training dataset in memory and there was absence of periodically validating our model using validation dataset in our code. In initializable iterator we overcome these problems. Initializable iterator has to be initialized with dataset before it starts running. Take a look at the code.

Code sample showing Initializable Iterator pipeline

As can be seen, using initializer operation, we have changed the dataset between training and validation using the same Dataset object.

This iterator is very ideal when you have to train your model with datasets which are split across multiple places and you are not able to accumulate them into one place.

c) Reinitializable

In initializable iterator, there was a shortfall of different datasets undergoing the same pipeline before the Dataset is fed into the iterator. This problem is overcome by reinitializable iterator as we have the ability to feed different types of Datasets thereby undergoing different pipelines. Only one care has to be taken is that different Datasets are of the same data type. Take a look at the code.

Code sample showing Reinitializable iterator pipeline

Notice that training Dataset object is undergoing additional augmentation which validation Dataset is not. You could have directly fed the training and validation datasets into Dataset objects but I have made use of placeholders just to show the flexibility.

d) Feedable

The reinitializable iterator gave the flexibility of assigning differently pipelined Datasets to iterator, but the iterator was inadequate to maintain the state (i.e till where the data has been emitted by individual iterator). In the code sample, I am showing how to use Feedable iterator.

Code sample showing Feedable Iterator pipeline

Though not illustrated in above code sample, using the string handle, we can restart the particular point from where the data extraction was done while altering between different Datasets.

This iterator is ideal in scenarios where you are training simultaneously a model with different datasets and you need better control to decide which particular batch of dataset has to be fed next to model.

Datasets can be generated with TFRecords as well. I have written another article on how to create TFRecords and how to feed it into Datasets in another article. Do check it out as it is like part-2 of this article.

How to use TFRecord with Datasets and Iterators in Tensorflow with code samples

In the previous article, I have demonstrated how to make use of Tensorflow's Datasets and Iterators. There we had...

[medium.com](#)

You can check the code used in this article from my Github repository.

Prasad9/TF_Dataset_Iterators

Illustration of how to use Tensorflow's Datasets and Iterators with MNIST image training...

[github.com](#)

Or if you wish to see the code running in notebook, you can check it in my shared Colab notebook.

Google Colaboratory

[DatasetIterator](#)

colab.research.google.com	
---------------------------	--

Do leave your valuable comments on what do you feel about this article.

TensorFlow

Data Science

Machine Learning

Artificial Intelligence

Data

About

Help

Terms

Privacy

Get the Medium app

